

PNM vs Baseline gem5 Comparison

PNM vs Baseline — gem5 Full-System Simulation Comparison

Hardware Configuration

Simulated Platform

Component	Baseline	PNM
ISA	x86-64	x86-64
OS	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS
Kernel	Linux 6.8.0-52-generic	Linux 6.8.0-52-generic
CPU model	SimpleSwitchable (KVM boot → TIMING ROI)	SimpleSwitchable (KVM boot → TIMING ROI)
CPU cores	1	2 (core 0 = writes, core 1 = compaction)
Clock	3 GHz	3 GHz
Memory	Dual-channel DDR4-2400, 3 GiB	Dual-channel DDR4-2400, 3 GiB
L1I cache	32 KiB (private)	32 KiB per core (private)
L1D cache	32 KiB (private)	32 KiB per core (private)
L2 cache	512 KiB (private)	512 KiB per core (private)
PNM unit	—	MMIO @ 0xD0000000, 500 ns latency, 25 GiB/s BW, DMA-enabled

The PNM unit is a custom gem5 PNMCompactor SimObject attached to the I/O bus. It receives compaction jobs from the guest via MMIO writes, models the compaction delay (process_latency + data transfer at 25 GiB/s), then DMA-writes a completion result back to DRAM.

Workload

Parameter	Value
Software	RocksDB v11.4.0, db_bench

Benchmark	fillrandom,waitforcompaction
Keys	15,000 × 16 B
Values	1,024 B (no compression)
Write buffer	1 MiB, max 2 buffers
L0 compaction trigger	2 files
Block size	4 KiB
Block cache	8 MiB
Max bytes for level base	16 MiB
Compaction style	Leveled (L0 → L1)
WAL	Disabled
Seed	42

The region of interest (ROI) is the fillrandom + waitforcompaction phase only. The kernel boot and RocksDB initialisation run under fast KVM before the ROI begins. The switch to detailed TIMING simulation happens at m5 workbegin.

1. Application Throughput — board.pc.com_1.device

This is the most important metric: what the workload actually observed.

Metric	Baseline	PNM	Improvement
fillrandom throughput	29,944 ops/sec	47,029 ops/sec	+57%
Latency per write	33.395 µs/op	21.197 µs/op	-36%
Total benchmark time	0.501 s	0.319 s	-36%
Write bandwidth	29.7 MB/s	46.6 MB/s	+57%

Why it improved: In baseline, the write thread stalls every time a compaction runs (L0→L1). In PNM, core 1 handles compaction concurrently via MMIO dispatch, so core 0's write thread never waits.

2. Compaction Behavior — board.pc.com_1.device

Baseline (sequential, blocking)

All compactations block the single write thread.

Job	Files	Elapsed
job=5	L0→L1, 2 files	102,985 µs
job=9	L0→L1, 4 files	199,969 µs
job=16	L0→L1, 7 files	198,970 µs
job=20	L0→L1, 5 files	137,979 µs
Total		~640 ms blocking

PNM (concurrent, MMIO-offloaded)

Core 1 runs compaction in parallel with core 0 writing. PNM hardware elapsed < DB elapsed because the PNM unit starts processing before RocksDB acknowledges the job.

Job	Files	PNM elapsed	DB elapsed	HW speedup
job $\approx 2 \times 10^9$	2 files	81,988 μ s	130,980 μ s	~20% faster
job $\approx 4 \times 10^{10}$	5 files	138,979 μ s	264,960 μ s	~30% faster
job $\approx 7 \times 10^{10}$	8 files	209,969 μ s	402,939 μ s	~5% faster
job $\approx 8 \times 10^{10}$	3 files	214,968 μ s	423,936 μ s	similar

PNM hardware completes compaction ~20-30% faster for smaller jobs and runs them **without blocking the write thread** — the dominant win.

3. Simulation-Level Statistics — stats.txt

Top-Level

Metric	Baseline	PNM	Note
simSeconds	5.642	5.998	+6.3% — PNM does more post-benchmark compaction
simInsts	1,069,152,478	1,462,561,905	+36.7% — 2 cores execute more total instructions
hostSeconds	2,446	3,296	+34.7% — 2nd core adds simulation overhead

The longer simulated time does **not** mean PNM is slower — it means the waitforcompaction phase drains more pending compactions, which the PNM unit handles concurrently.

CPU Performance (TIMING mode)

Metric	Baseline (1 core)	PNM core 0 (writes)	PNM core 1 (compaction)
IPC	0.062890	0.043547	0.037445
CPI	15.900898	22.963557	26.705966
Cycles	16.9 B	18.0 B	18.0 B

Per-core IPC drops in PNM because both cores share the same DDR4 bus, adding memory-bandwidth contention. Total throughput still increases because compaction and writes now overlap.

L1D Cache (core 0)

Metric	Baseline	PNM core 0	Change
demandMisses	6,539,396	4,744,036	-27.5%
demandMissRate	1.662%	1.841%	higher rate, fewer total accesses
demandAvgMissLatency	14,287 ticks	10,670 ticks	-25.3%

Core 0 in PNM touches no compaction data (SST merging, key sorting). Removing that sequential-scan pollution reduces both L1D misses and average miss latency.

L2 Cache (core 0)

Metric	Baseline	PNM core 0	Change
demandHits	40,212,372	23,593,459	-41.3%
demandMisses	4,349,508	2,209,150	-49.2%
demandMissLatency (ticks)	212,789,079,395	105,792,142,897	-50.3%

Core 0's L2 miss pressure is **halved** because compaction I/O no longer evicts the RocksDB write-path working set. PNM adds a separate l2-cache-1 for core 1's compaction traffic.

Memory Controller (mem_ctrl0)

Metric	Baseline	PNM	Change
readReqs	3,509,925	3,261,016	-7.1%
writeReqs	1,330,691	1,270,170	-4.5%
bytesReadSys	224.6 MB	208.7 MB	-7.1%
bytesWrittenSys	85.2 MB	81.3 MB	-4.6%
avgRdBWSys	39.8 MB/s	34.8 MB/s	-12.6% per controller
avgWrBWSys	15.1 MB/s	13.6 MB/s	-9.9% per controller

Per-controller traffic is slightly lower because core 0 no longer performs compaction I/O. PNM has two memory controllers (split DRAM channels), so combined bandwidth is higher overall.

4. Summary

Dimension	Effect	Mechanism
Write throughput	+57%	Compaction runs on core 1 concurrently; write thread never stalls
Write latency	-36%	No write-thread blocking for compaction
L1D cache efficiency	-25% miss latency	Compaction scan no longer pollutes core 0's L1D
L2 cache efficiency	-50% miss latency	Write-path working set fits L2 without compaction evictions

Compaction HW speed	~20-30% faster	PNM unit finishes L0→L1 merges faster for small jobs
Memory bandwidth	Neutral/improved	Traffic split across 2 controllers; per-CPU traffic reduced
Per-core IPC	-31%	Expected: 2 cores share one DDR4 bus (bandwidth-limited)

Bottom line: The dominant benefit is **concurrency** — compaction no longer serialises with writes. Cache-pollution elimination is a bonus that further reduces write-path latency. The PNM hardware being intrinsically faster is secondary.